PRASAD V POTLURI SIDDHARTHA INSTITUTE OF TECHNOLOGY
DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

ACADEMIC YEAR: 2026-27

DEEP LEARNING COURSE PROJECT REPORT

PHISHING URL DETECTION USING CHARACTERLEVEL LSTM NEURAL NETWORKS

Faculty Name: Dr. G. Lalitha Kumari

Team Members:

1. **Mandadhi Janani 23501A05B2**
2. **Mukku Dakshayani 23501A05C5**
3. **Muddineni Yamini 23501A05C3**

ABSTRACT

Phishing is a cyberattack in which attackers create deceptive websites and URLs to trick users into revealing sensitive information, such as usernames, passwords, banking details, and personal data. Traditional security methods may fail to identify newly created phishing URLs because attackers frequently modify URL structures to avoid detection.

This project presents a phishing URL detection system using a character-level Long Short-Term Memory (LSTM) neural network. The system uses the PhiUSIIL Phishing URL Dataset obtained through the UCI Machine Learning Repository. The dataset is processed to extract URL text and corresponding class labels, where legitimate URLs are represented by class 0 and phishing URLs by class 1.

Each URL is converted into a sequence of numerical character IDs. The sequences are padded or truncated to a fixed length of 200 characters and supplied to an LSTM-based deep learning model. The model contains an embedding layer, spatial dropout, an LSTM layer, a dense hidden layer, dropout, and a sigmoid output layer.

The dataset is divided into training, validation, and testing sets. Model performance is evaluated using accuracy, precision, recall, F1-score, ROC-AUC, and a confusion matrix. The trained model can also classify a user-provided URL and estimate its phishing probability.

Another important aspect of this project is the ability of LSTM networks to learn sequential relationships between characters in a URL. Unlike traditional machine learning methods that often depend on predefined features, a character-level model processes the URL as a sequence and learns patterns during training. These patterns may include unusual character combinations, suspicious subdomains, misleading words, unexpected symbols, and URL structures commonly associated with phishing attempts. This approach can help the system recognize patterns that are difficult to identify through simple rule-based techniques. However, the effectiveness of the model depends on the quality, diversity, and representativeness of the training dataset.

KEYWORDS: *Phishing URL Detection, Deep Learning, Long Short-Term Memory (LSTM), Character-Level Encoding, Cybersecurity, URL Classification, PhiUSIIL Dataset.*

INTRODUCTION

The internet is widely used for online banking, shopping, education, communication, social networking, and other digital activities. As internet usage increases, cyber threats also continue to grow. One common cyber threat is phishing, which involves deceiving users into visiting fraudulent websites or providing confidential information to attackers.

Phishing attacks often use URLs that resemble the addresses of trusted websites. Attackers may include misleading words, unusual subdomains, additional characters, or modified domain names to make a malicious URL appear genuine. A user who fails to identify these differences may unknowingly disclose sensitive information.

Traditional phishing detection techniques commonly rely on blacklists, manually designed URL features, domain information, or website content. Although these techniques are useful, they may not always recognize newly generated or modified phishing URLs.

Deep learning provides another approach to phishing detection by learning patterns from data. In this project, a character-level LSTM neural network is used to analyze the sequence of characters in a URL. Instead of depending entirely on manually extracted features, the model learns character patterns that may help distinguish phishing URLs from legitimate URLs.

The system first collects and preprocesses URL data, converts the characters into numerical sequences, trains the LSTM model, and evaluates its performance on unseen test data. Finally, it accepts a URL entered by the user and produces a predicted class and estimated phishing probability.

Furthermore, the proposed system focuses on making phishing detection more accessible by allowing users to check URLs without requiring detailed technical knowledge. By learning from the patterns present in the training dataset, the LSTM model can identify similarities and differences between legitimate and phishing URLs. The system's performance is assessed using multiple evaluation metrics to understand its reliability and limitations. This approach can serve as a useful foundation for developing more advanced cybersecurity applications that help users recognize suspicious links and make safer decisions while browsing the internet.

PROBLEM STATEMENT

Phishing attacks use deceptive URLs to mislead users and obtain confidential information. Identifying such URLs is challenging because attackers can frequently modify their structure and appearance to imitate legitimate websites. Conventional approaches based on known malicious URL lists or manually selected features may not identify every new or modified phishing URL.

Therefore, there is a need for an automated system that can learn character-level patterns from URL data and classify URLs as phishing or legitimate. The proposed project addresses this problem by developing a character-level LSTM neural network that processes URL sequences and predicts whether a given URL resembles a phishing URL or a legitimate URL.

Objectives of the Project

The main objectives of this project are:

1. To develop an automated system for classifying URLs as phishing or legitimate.
2. To obtain URL data from the PhiUSIIL Phishing URL Dataset available through the UCI Machine Learning Repository.
3. To clean and preprocess the URL data and prepare the corresponding binary labels.
4. To convert URL characters into numerical sequences suitable for deep learning.
5. To build an LSTM neural network capable of learning sequential character patterns.
6. To train the model using training data and monitor its performance using validation data.
7. To evaluate the trained model using accuracy, precision, recall, F1-score, ROC-AUC, and a confusion matrix.
8. To provide a function that accepts a new URL and predicts its class and estimated phishing probability.
9. To save the trained model and preprocessing configuration for future use.

LITERATURE REVIEW

Phishing URL detection has been studied using traditional machine learning, rule-based techniques, blacklist-based methods, and deep learning approaches.

Blacklist-based detection: This approach compares a URL against a database of known malicious addresses. It is simple and useful for previously identified threats, but newly generated phishing URLs may not be present in the database.

Feature-based machine learning: These approaches extract properties such as URL length, the number of special characters, domain characteristics, and the presence of suspicious words. A machine learning classifier then uses these properties to predict the URL class. Their performance depends on the selected features and the quality of the data.

Deep learning-based detection: Deep learning models can learn complex patterns from training data. Models that process URL text can identify patterns without relying exclusively on manually designed features.

Character-level sequence models: Character-level models convert a URL into a sequence of characters and learn patterns in their order. LSTM networks are designed to process sequences and retain useful information across multiple time steps.

Based on these approaches, this project implements a character-level LSTM model. It uses the PhiUSIIL dataset and learns from URL character sequences to perform binary classification. A formal comparative study against other algorithms would require implementing and evaluating those models separately.

Note: This section provides general background. Your notebook does not include a complete bibliography or experiments comparing multiple published methods, so add verified research papers if your college requires formal literature-review references.

EXISTING SYSTEM

Existing phishing detection systems use several approaches to identify suspicious URLs. These include blacklists of known phishing websites, rule-based filtering, URL feature extraction, and conventional machine learning classifiers.

Blacklist-based systems can identify URLs that have already been reported, but they may struggle with newly created phishing websites. Rule-based systems depend on predefined conditions and may not cover every URL pattern. Feature-based machine learning systems require suitable features to be selected and extracted before classification.

These approaches are useful, but their effectiveness depends on the detection method, available data, and changing attack patterns. This project explores a character-level deep learning approach that learns URL patterns directly from the URL text.

LIMITATIONS OF THE EXISTING SYSTEM

The following are common limitations of existing approaches:

1. Dependence on known URLs: Blacklists may not recognize previously unseen phishing URLs.
2. Manual feature selection: Feature-based systems require suitable URL characteristics to be identified and extracted.
3. Changing attack patterns: Attackers can modify URLs to avoid simple detection rules.
4. Limited generalization: A model trained on one dataset may not perform equally well on URLs from different sources or time periods.
5. Maintenance requirements: Blacklists and predefined rules need regular updates.
6. False predictions: A legitimate URL may be incorrectly flagged, or a phishing URL may be missed.

These limitations motivate the exploration of deep learning techniques that learn patterns from URL sequences. However, the proposed LSTM system can also make errors and must be evaluated using appropriate test data.

PROPOSED SYSTEM

The proposed system uses a character-level LSTM neural network to classify URLs as legitimate or phishing. The PhiUSIIL dataset is loaded, and the URL text and corresponding labels are extracted. Missing or empty records are removed, and the labels are converted into a binary format.

The URLs are transformed into numerical character sequences using a character vocabulary. Each sequence is padded or truncated to 200 characters. The prepared data is divided into training, validation, and testing sets while preserving class proportions.

An LSTM model is then constructed using an embedding layer, spatial dropout, an LSTM layer, a dense layer, dropout, and a sigmoid output layer. The model is trained using binary cross-entropy loss and the Adam optimizer. Early stopping and learning-rate reduction callbacks help manage training.

After training, the model is evaluated using a separate test set. The system calculates classification metrics and generates visualizations such as the confusion matrix and ROC curve. A prediction function is also provided to classify a new URL entered by the user.

ADVANTAGES OF THE PROPOSED SYSTEM

1. Learns character patterns directly from URL text.
2. Reduces dependence on manually designed URL features.
3. Supports binary classification of legitimate and phishing URLs.
4. Produces an estimated phishing probability.
5. Uses separate training, validation, and test datasets for model development and evaluation.
6. Provides performance measures that help assess classification quality.
7. Saves the trained model and character vocabulary for later use.
8. Can be extended into a web application or integrated with a larger security system.

SYSTEM REQUIREMENTS

Hardware requirements

- Computer or laptop
- Processor capable of running Python and TensorFlow
- Sufficient RAM for dataset processing and model training
- Internet connection for downloading the dataset and installing dependencies

Software requirements

- Python
- Google Colab or a compatible Jupyter Notebook environment
- TensorFlow and Keras
- NumPy
- Pandas
- Matplotlib
- Seaborn
- Scikit-learn
- ucimlrepo dataset loader

Programming language: Python

Deep learning framework: TensorFlow/Keras

Dataset source: PhiUSIIL Phishing URL Dataset through the UCI Machine Learning Repository

DATASET DESCRIPTION

This project uses the PhiUSIIL Phishing URL Dataset, obtained through the UCI Machine Learning Repository. The notebook retrieves the dataset using the fetch_ucirepo function with dataset ID 967.

The dataset contains input features and target labels. The implementation combines these two components, identifies the URL column and the label column, and creates a smaller dataset containing the URL text and its corresponding class.

Records with missing URL values or missing labels are removed. URLs are converted to strings and surrounding whitespace is removed. The target labels are mapped to two classes:

- Class 0: Legitimate URL
- Class 1: Phishing URL

To keep model training manageable, the notebook sets a maximum sample count of 100,000 records. If the cleaned dataset contains more than this number, the notebook samples records while attempting to preserve the original class proportions.

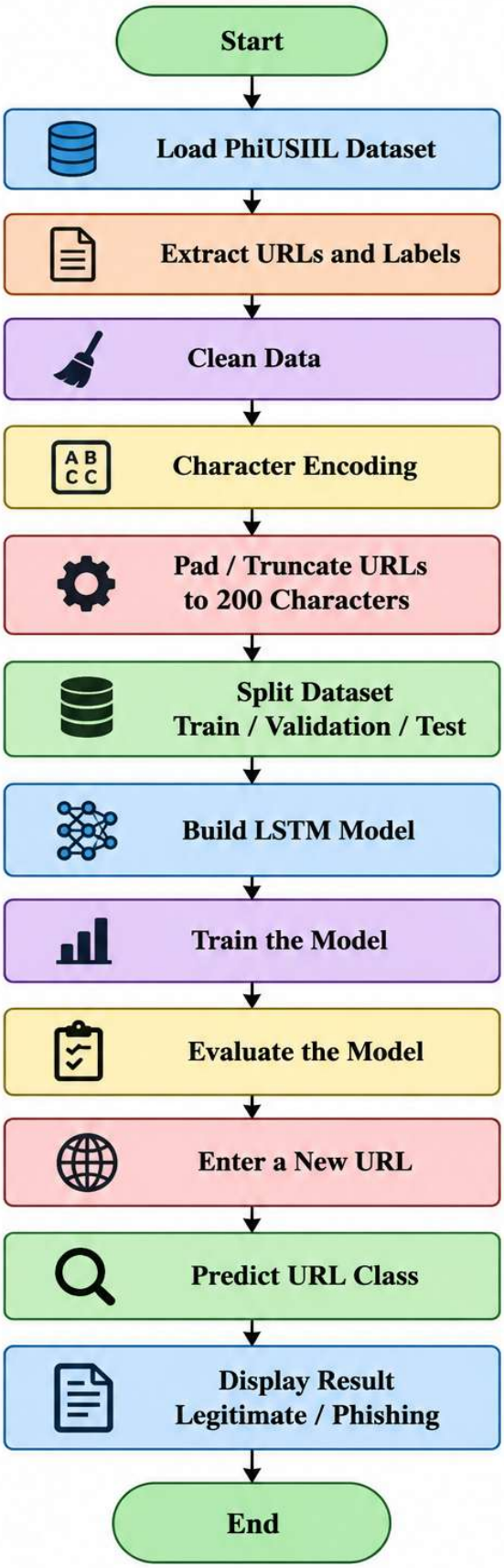
The final number of usable records depends on the dataset version and the number of records remaining after cleaning. The actual dataset dimensions and class counts should be taken from the notebook execution output.

METHODOLOGY

The project follows a sequence of steps, from dataset collection to URL prediction.

1. Download the PhiUSIIL dataset
2. Extract URLs and binary labels
3. Clean and explore the dataset
4. Encode characters into numerical IDs
5. Pad or truncate URLs to 200 characters
6. Split into training, validation and test sets
7. Build and train the LSTM model
8. Evaluate the model performance
9. Predict the class of a new URL

FLOW CHART:


Start
Load PhiUSIIL Dataset
Extract URLs and Labels
Clean Data
Character Encoding
Pad / Truncate URLs to 200 Characters
Split Dataset Train / Validation / Test
Build LSTM Model
Train the Model
Evaluate the Model
Enter a New URL
Predict URL Class
Display Result Legitimate / Phishing
End

DATA PREPROCESSING

Data preprocessing converts raw URL information into a consistent format that can be used by the LSTM model.

First, the notebook removes records with missing URLs or labels and removes whitespace surrounding each URL. It then converts the URL values into strings and prepares the binary labels.

Next, the cleaned URLs are converted into numerical sequences. A character vocabulary is constructed, and each character is assigned a unique integer ID. ID 0 is reserved for padding, while ID 1 represents unknown characters.

Since URLs have different lengths, the sequences are adjusted to a fixed length of 200 characters. Short URLs are padded at the end with zeros, and longer URLs are truncated after the maximum length.

This process produces a numerical input matrix that can be passed to the neural network.

CHARACTER-LEVEL ENCODING

Character-level encoding represents each character in a URL using a numerical ID. For example, a URL containing letters, numbers, dots, slashes, and other symbols is converted into a sequence of integer values.

The model does not directly process the original text as ordinary words. Instead, it receives the numerical representation of the URL characters.

Illustrative example:

http://example.com

is represented conceptually as:

h → 12, t → 8, t → 8, p → 5, : → 3, / → 7, ...

These numbers are only illustrative, not the actual character IDs assigned by the notebook.

Character-level processing allows the model to learn patterns in domain names, separators, suspicious strings, and the ordering of characters. It does not require the URL to be divided into complete words before training.

DATASET SPLITTING

The notebook divides the prepared dataset into three subsets:

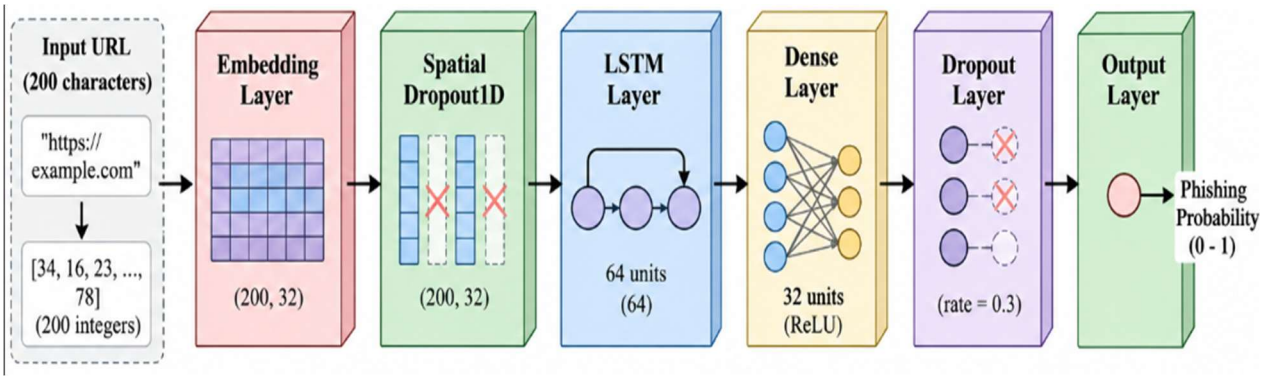
- Training set: 80% of the data, used to learn model parameters.
- Validation set: 10% of the data, used to monitor performance during training.
- Testing set: 10% of the data, used for final evaluation on unseen samples.

The first split reserves 20% of the data for validation and testing. This portion is then divided equally into two subsets. Stratified splitting is used to preserve the class proportions as closely as possible.

The notebook uses a fixed random seed to improve reproducibility. The actual number of records in each subset is printed during execution.

LSTM Model Architecture

The proposed model uses the following architecture:



EXPLANATION OF THE LAYERS

1. Input layer: Accepts the encoded URL as a sequence of 200 integers.

2. Embedding layer: Converts each character ID into a learnable vector of 32 values. This allows the network to learn useful representations of different characters.

3. SpatialDropout1D layer: Randomly drops embedding channels during training to help reduce overfitting.

4. LSTM layer: Processes the sequence of character embeddings and learns relationships between characters. It contains 64 units and uses a dropout rate of 0.2.

5. Dense layer: Contains 32 neurons with the ReLU activation function. It learns additional patterns from the LSTM representation.

6. Dropout layer: Uses a dropout rate of 0.3 to reduce reliance on particular activations during training.

7. Output layer: Contains one neuron with sigmoid activation. Its output is a value between 0 and 1 that the system interprets as the estimated probability of phishing.

MODEL TRAINING

The LSTM model is trained using the training dataset, while the validation dataset is used to monitor learning performance.

The notebook uses the Adam optimizer to update model parameters and binary cross-entropy as the loss function. Binary cross-entropy is suitable for a classification task with two classes.

The training configuration specifies a maximum of 10 epochs and a batch size of 128. An epoch represents one complete pass through the training dataset, while the batch size determines how many samples are processed together before a parameter update.

Two callbacks are used during training. Early stopping monitors validation loss and stops training when it fails to improve for two consecutive epochs, restoring the best model weights. ReduceLROnPlateau reduces the learning rate when validation loss stops improving.

These settings help manage training time and reduce the risk of overfitting. The final number of epochs actually completed depends on the training behavior.

IMPLEMENTATION DETAILS

The project is implemented in Python using TensorFlow/Keras for deep learning and Scikit-learn for dataset splitting and evaluation.

The implementation is divided into the following modules:

1. Library installation and imports: Loads the required Python packages.
2. Dataset loading: Retrieves the PhiUSIIL dataset using the UCI dataset loader.
3. Data preparation: Selects URL text and class labels, removes missing values, and cleans the data.

4. Exploratory data analysis: Visualizes class distribution and URL-length patterns.
5. Character encoding: Builds a vocabulary and converts URLs into numerical sequences.
6. Data splitting: Creates training, validation, and test sets.
7. Model construction: Builds the embedding-based LSTM neural network.
8. Model training: Trains the network using validation monitoring.
9. Performance evaluation: Calculates classification metrics and plots the confusion matrix and ROC curve.
10. URL prediction: Accepts a URL from the user and returns the predicted class and estimated phishing probability.
11. Model saving: Stores the trained network, character vocabulary, and preprocessing configuration.

ALGORITHM

Input: A labeled dataset containing URL text and class labels.

Output: A trained LSTM model that predicts whether a URL is phishing or legitimate.

Algorithm steps:

1. Start the program.
2. Import the required libraries.
3. Download the PhiUSIIL dataset.
4. Dataset: https://archive.ics.uci.edu/dataset/967/phiusiil%2Bphishing%2Burl%2Bdataset
5. Extract URL values and target labels.
6. Remove records with missing or empty URLs and missing labels.
7. Assign class 0 to legitimate URLs and class 1 to phishing URLs.
8. Limit the dataset to a maximum of 100,000 samples when necessary.
9. Construct a character vocabulary and encode URLs into numerical sequences.
10. Pad or truncate each sequence to 200 characters.

11. Divide the data into training, validation, and test sets using stratified splitting.
12. Build the embedding-based LSTM model.
13. Compile the model using the Adam optimizer and binary cross-entropy loss.
14. Train the model for up to 10 epochs with early stopping and learning-rate reduction.
15. Predict class probabilities for the test dataset.
16. Convert probabilities to classes using a threshold of 0.5.
17. Calculate accuracy, precision, recall, F1-score, and ROC-AUC.
18. Display the confusion matrix and ROC curve.
19. Accept a new URL and predict its class.
20. Save the trained model and preprocessing files.
21. Stop the program.

PERFORMANCE EVALUATION METRICS

The project uses several evaluation metrics to measure the quality of phishing URL classification.

1. Accuracy: Measures the proportion of all URLs correctly classified.

2. Precision: Measures how many URLs predicted as phishing are actually phishing URLs.

3. Recall: Measures how many actual phishing URLs are correctly identified.

4. F1-score: Combines precision and recall into a single metric using their harmonic mean.

5. ROC-AUC: Measures how well the model distinguishes the two classes across different classification thresholds. A higher value generally indicates better class separation.

6. Confusion matrix: Shows correct and incorrect predictions for both legitimate and phishing URLs.

Here, TP means true positives, TN means true negatives, FP means false positives, and FN means false negatives.

RESULTS AND DISCUSSION

The project evaluates the trained character-level LSTM model using a separate test dataset. The model generates phishing probabilities for the test URLs, and a threshold of 0.5 is used to convert the probabilities into binary predictions.

The evaluation module calculates accuracy, precision, recall, F1-score, and ROC-AUC. It also generates a classification report, confusion matrix, and ROC curve. Training and validation performance are visualized to help understand how the model learns and whether overfitting may occur.

The confusion matrix provides additional insight into the types of classification errors. False positives represent legitimate URLs incorrectly identified as phishing, while false negatives represent phishing URLs incorrectly classified as legitimate.

The final performance of the model must be reported using the actual output produced when the notebook runs. These values should be copied directly from the evaluation results rather than estimated or assumed.

Use this box to insert the actual results after running your notebook:

MODEL EVALUATION RESULTS

```
Model Evaluation Results:
-------------------------
Accuracy  : 0.9988 (99.88%)
Precision : 1.0000 (100.00%)
Recall    : 0.9972 (99.72%)
F1-score  : 0.9986
ROC-AUC   : 0.9991

Confusion Matrix Values:
------------------------
True negatives : 5730 (Legitimate correctly classified)
False positives: 0 (Legitimate incorrectly flagged)
False negatives: 12 (Phishing incorrectly missed)
True positives : 4258 (Phishing correctly detected)
```

APPLICATIONS

1. Email security: Can support systems that screen URLs contained in emails for potential phishing.
2. Web browsing protection: Can be integrated into a larger browser security solution to flag suspicious URLs.
3. Cybersecurity awareness: Can demonstrate why URLs should be checked carefully before users click them.
4. Organizational security: Can serve as a component in a larger security workflow for reviewing suspicious links.
5. Security research and education: Can be used to study character-level deep learning and binary classification.
6. Phishing analysis tools: Can support applications that prioritize suspicious URLs for further investigation.

These are potential applications. The notebook implements model-based URL prediction but does not itself provide an email scanner, browser extension, or live website protection system.

ADVANTAGES

- Processes URL text directly at the character level.
- Learns sequential patterns using an LSTM network.
- Does not require a separate manually designed URL feature set for the implemented model.
- Uses validation data to monitor training.
- Applies dropout and early stopping to help control overfitting.
- Evaluates performance using multiple complementary metrics.
- Supports predictions on user-entered URLs.
- Allows the trained model and character vocabulary to be saved and loaded later.

LIMITATIONS

- The model can produce false positive and false negative predictions.
- The results depend on the quality, distribution, and coverage of the training dataset.
- URLs longer than 200 characters are truncated, so information after that point is not available to the model.
- The implementation does not inspect webpage content, domain registration information, certificates, or live website behavior.
- The model may perform differently on newly emerging phishing patterns or data from different sources.
- Training a deep learning model may require considerable time and memory, depending on the hardware and dataset size.
- The predicted probability is a model estimate and is not necessarily a calibrated measure of real-world risk.
- The system should not be used as the sole basis for deciding whether a website is safe.

FUTURE ENHANCEMENTS

The project can be extended in several ways:

1. Web application development: Create a simple web interface where users can enter a URL and view the prediction.
2. Browser extension: Integrate the model into a browser extension to flag suspicious URLs.
3. Dataset expansion: Train and evaluate the model on additional, diverse, and more recent URL datasets.
4. Model comparison: Compare LSTM performance against CNN, GRU, Random Forest, and other suitable classification models.
5. Advanced URL analysis: Combine character-level analysis with domain information, URL lexical features, and website reputation data.
6. Model optimization: Investigate model compression and inference optimization to reduce prediction time.
7. Improved evaluation: Test on temporally separated or externally sourced data to study generalization to new phishing attacks.

8. Explainability: Add methods to help users understand which URL patterns influenced a prediction.
9. Security integration: Integrate the model with an email filtering or security monitoring workflow.
10. Threshold tuning: Select the classification threshold based on the relative costs of false positives and false negatives.

CONCLUSION

This project presents a character-level LSTM neural network for phishing URL detection. The system uses the PhiUSIIL Phishing URL Dataset, performs data cleaning and preprocessing, converts URLs into numerical character sequences, and trains an LSTM-based model to classify URLs as legitimate or phishing.

The model architecture includes an embedding layer, spatial dropout, an LSTM layer, dense layers, dropout, and a sigmoid output layer. The model is trained using the Adam optimizer and binary cross-entropy loss, with validation-based callbacks to manage training.

The implementation evaluates the trained model using accuracy, precision, recall, F1-score, ROC-AUC, and a confusion matrix. It also provides a function for predicting the class of a user-entered URL and saving the trained model for future use.

The project demonstrates how deep learning can be applied to URL-based cybersecurity tasks. Although the system can assist in identifying potentially suspicious URLs, its predictions are not definitive security verdicts. Further testing on diverse and recent datasets, along with integration into a broader security framework, would be necessary before practical deployment.

REFERENCES

1. UCI Machine Learning Repository, PhiUSIIL Phishing URL Dataset. Available at: https://archive.ics.uci.edu/dataset/967/phiusiil+phishing+url+dataset.
2. TensorFlow, TensorFlow Documentation. Available at: https://www.tensorflow.org/.
3. Keras, Keras API Documentation. Available at: https://keras.io/.
4. Scikit-learn, Machine Learning in Python. Available at: https://scikit-learn.org/.